"The Central Hospital" Web Portal

Course: Introduction to Web Technology LAB

Student Name: Tarek Rajab

Roll No: 23BCP339

Instructor: Nandini Modi

Date: November 10, 2025

Table of Contents

1. Introduction

1.1 Project Objective

The primary objective of this project was to design, develop, and deploy a functional, multi-page website for a fictional hospital named "The Central Hospital." The goal was to create a professional, user-friendly, and informative web portal that serves as the primary point of contact for patients and visitors.

The project aimed to demonstrate proficiency in full-stack development by integrating a static frontend (HTML, CSS) with dynamic, client-side functionality (JavaScript) and a persistent backend (PHP, MySQL) for handling user-submitted data.

1.2 Scope and Functionality

The website includes the following core features:

- **Informational Pages:** Static content pages providing details about the hospital, its services, and its staff.
- **Doctor Search:** A client-side search feature for patients to find doctors by name or department.
- **Contact Form:** A functional contact form that captures user messages and stores them in a database.
- **Appointment Booking:** A dynamic, multi-step appointment booking system that validates user input and saves appointments to a database.

1.3 Technologies Used

- **Frontend:** HTML5, CSS3, JavaScript (ES6+), Tailwind CSS (for the booking page)
- **Backend:** PHP 8
- **Database:** MySQL
- **Environment:** XAMPP (Apache, MySQL, PHP)

2. System Architecture

The project follows a standard client-server architecture.

2.1 Frontend Architecture

The frontend consists of 7 HTML pages, styled using internal CSS (<style> blocks). Most pages are static, with JavaScript used to provide interactivity on specific pages.

- **Home.html, AboutUs.html, OurServices.html, ContactUs.html:** Primarily static content pages.
- **FindADoctor.html:** Uses client-side JavaScript to filter a hardcoded JSON array of doctors, demonstrating DOM manipulation.
- **SendUsAMessage.html:** A standard HTML form that submits data to the PHP backend.
- **BookAnAppointment.html:** The most complex frontend page. It uses Tailwind CSS for styling and features a multi-step form managed entirely by JavaScript. It uses the fetch API to send data to the backend without a page reload (asynchronously). It also uses localStorage to simulate "booked" time slots.

2.2 Backend Architecture

The backend is built with PHP and handles two specific functions: processing contact messages and booking appointments. It acts as an API for the frontend, receiving POST requests, processing data, interacting with the database, and returning a status.

2.3 Database Design

The MySQL database, named central_hospital, contains two main tables:

1. **messages**: Stores submissions from the SendUsAMessage.html form.
    - id (INT, PK, AI)
    - first_name (VARCHAR)
    - last_name (VARCHAR)
    - email (VARCHAR)

- phone (VARCHAR)
- reason (VARCHAR)
- message (TEXT)
- created_at (TIMESTAMP)

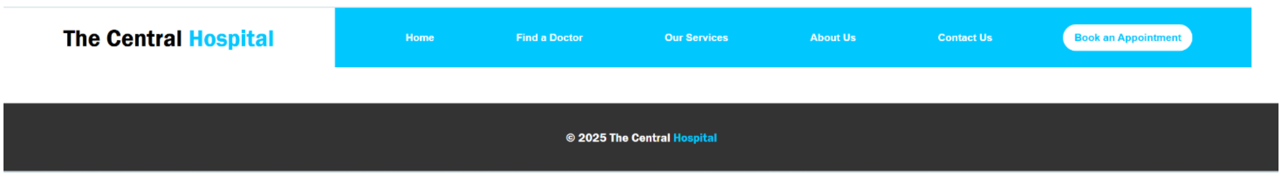
2. **appointments**: Stores submissions from the BookAnAppointment.html form.
    - id (INT, PK, AI)
    - patient_name (VARCHAR)
    - phone (VARCHAR)
    - email (VARCHAR)
    - blood_type (VARCHAR)
    - department (VARCHAR)
    - appointment_date (VARCHAR)
    - appointment_time (VARCHAR)
    - created_at (TIMESTAMP)

3. Frontend Implementation: Page-by-Page

3.1 Common Elements: Header and Footer

A consistent header and footer are present on all pages. The header is built with position: fixed and split into two parts:

- **.div1**: The hospital logo/name on the left.
- **.div2**: The navigation bar on the right, which uses flexbox to distribute links evenly. The footer is a simple, static element with the copyright notice.



3.2 Home Page (Home.html)

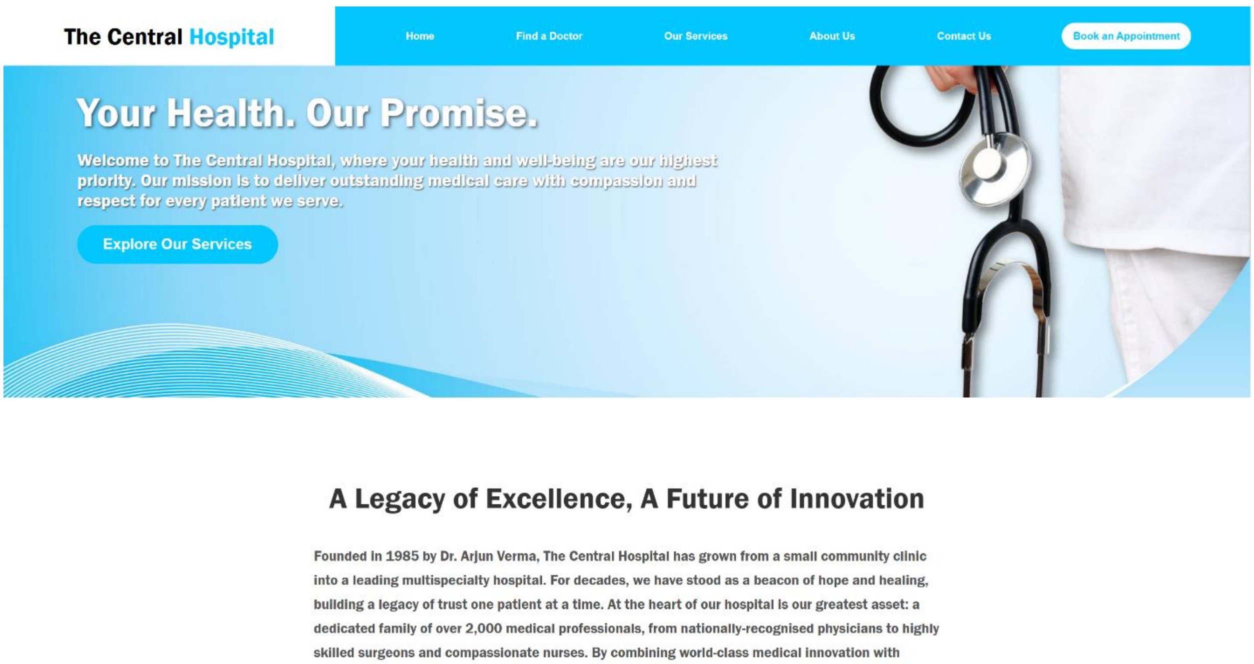
This is the landing page for the site.

- **Hero Banner (.div3)**: A full-height banner with a background image and a prominent "Book an Appointment" call-to-action (CTA) button.
- **About Snippet (.div6)**: A brief introduction to the hospital.
- **Statistics Section (.div4)**: Uses flexbox to display three cards (.div5) highlighting key metrics (experts, beds, research projects).

3.3 About Us Page (AboutUs.html)

This page expands on the hospital's identity.

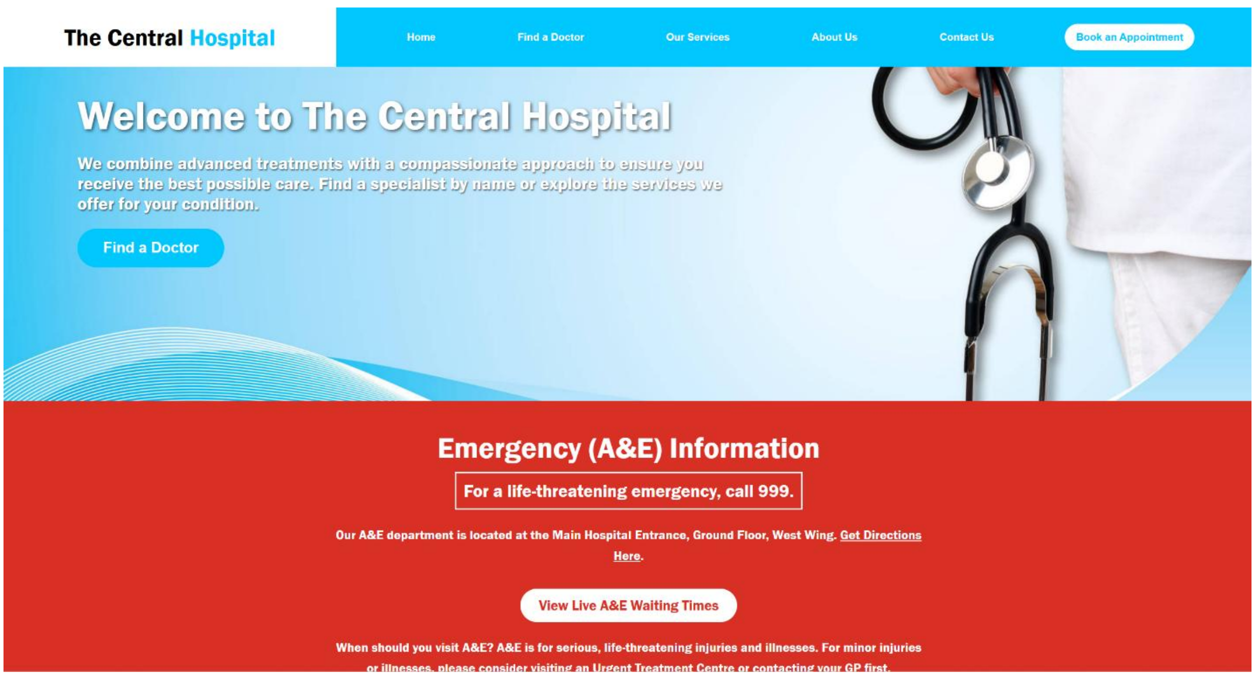
- It includes a hero banner, similar to the home page but with different text.
- The main content section (.div6) provides a more detailed history.
- It features three "value cards" (Compassion, Excellence, Innovation) to highlight the hospital's core principles.



3.4 Our Services Page (OurServices.html)

This page is a comprehensive hub for patient information.

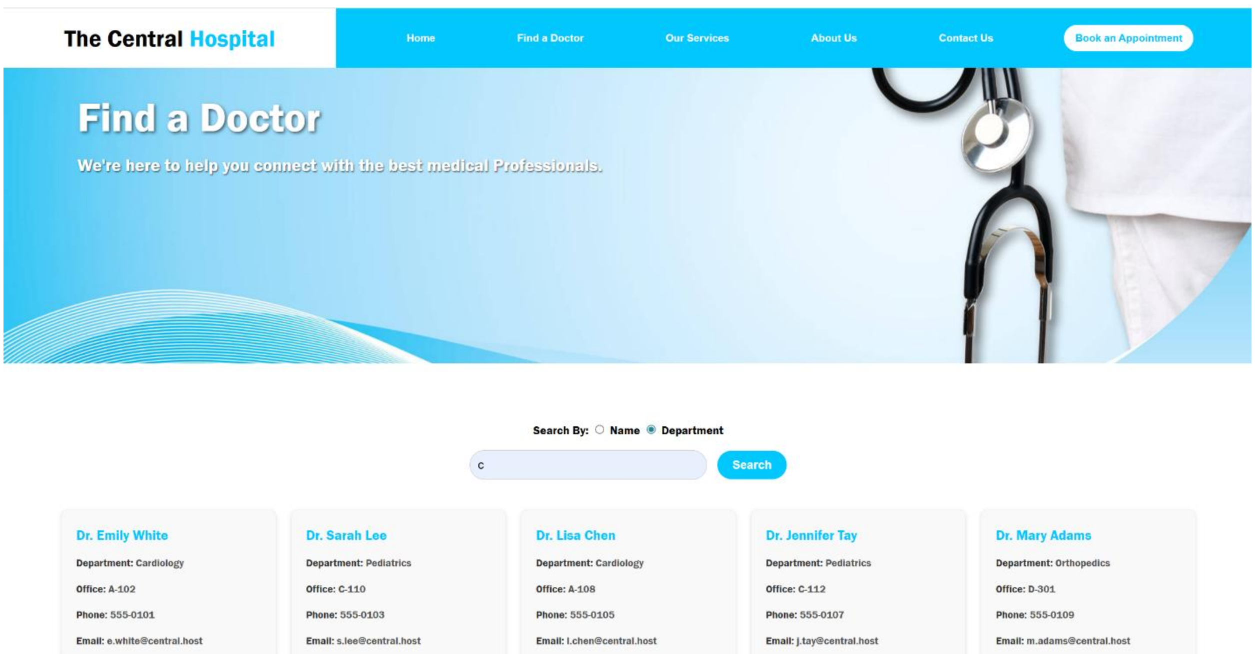
- **Emergency Section:** A prominent, red-colored section providing critical emergency information.
- **Department List:** A large, alphabetized list of all hospital departments, with each item intended to be a link.
- **Patient Stories:** A section using "story cards" to build social proof and trust.
- **Patient & Visitor Info:** A grid-based layout (.info-grid) with cards for practical information like visiting hours, maps, and facilities.



3.5 Find a Doctor Page (FindADoctor.html)

This page provides a dynamic search function.

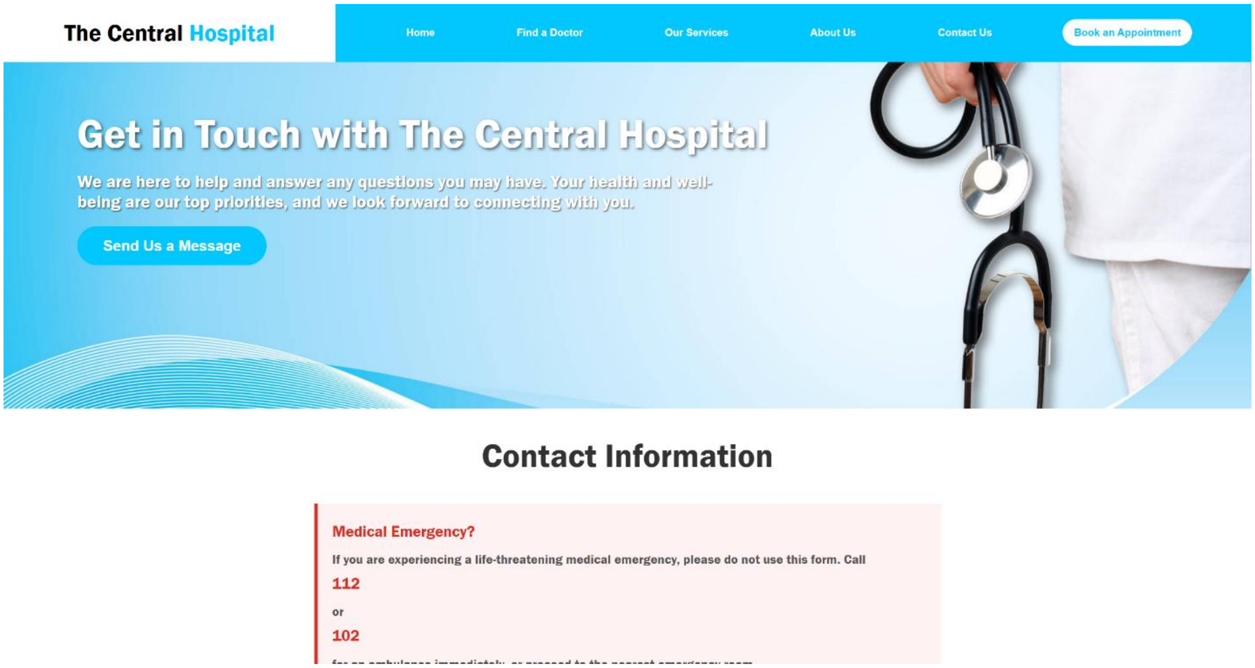
- **Search Bar:** Allows users to search by "Name" or "Department" using radio buttons.
- **JavaScript Logic:** A script block contains a hardcoded array of doctors. An event listener on the "Search" button triggers the performSearch function.
- **performSearch()**: This function filters the doctors array based on the search term and selected radio button. It then dynamically creates "doctor-card" elements and injects them into the SearchResults div, demonstrating DOM manipulation.

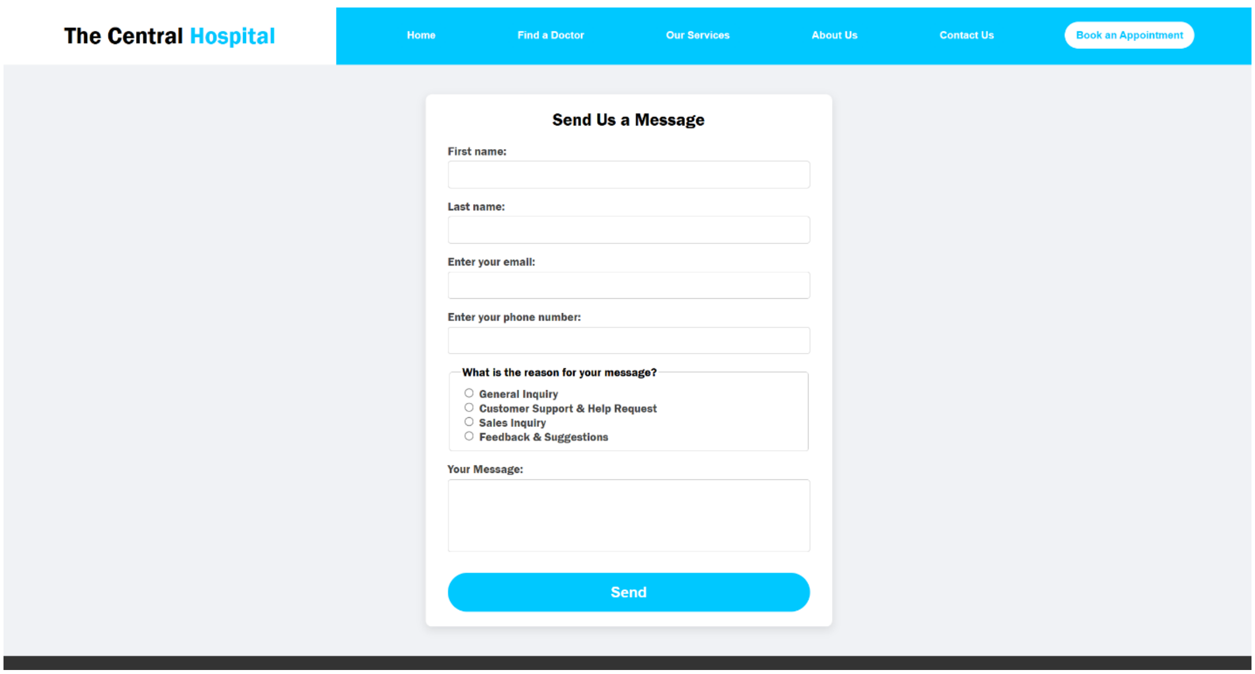


3.6 Contact & Message Pages (ContactUs.html & SendUsAMessage.html)

These two pages work together.

- **ContactUs.html**: This is a static page that provides physical address, phone numbers, and an emergency notice. It links to the message form.
- **SendUsAMessage.html**: This page contains the actual HTML form. The form uses the POST method and has the action="contact.php" attribute, which sends the data to the backend script when submitted. It includes basic form validation like required attributes.

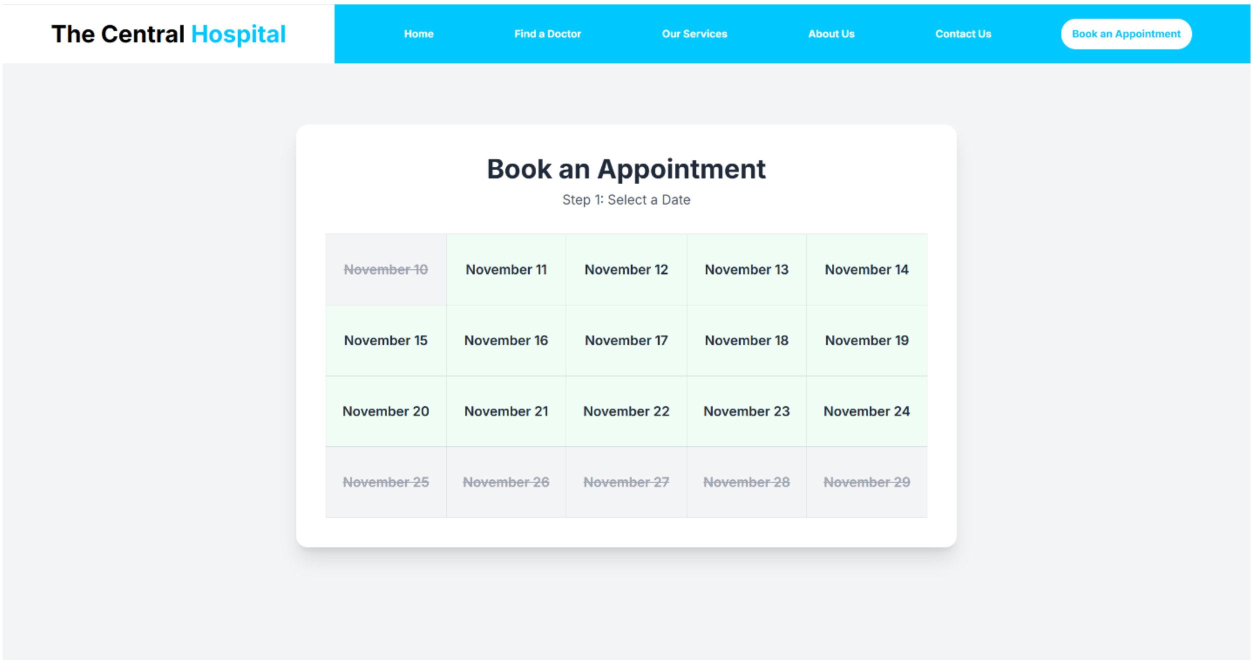

The Central Hospital
Home
Find a Doctor
Our Services
About Us
Contact Us
Book an Appointment
Get in Touch with The Central Hospital
We are here to help and answer any questions you may have. Your health and well-being are our top priorities, and we look forward to connecting with you.
Send Us a Message
Contact Information
Medical Emergency?
If you are experiencing a life-threatening medical emergency, please do not use this form. Call
112
or
102

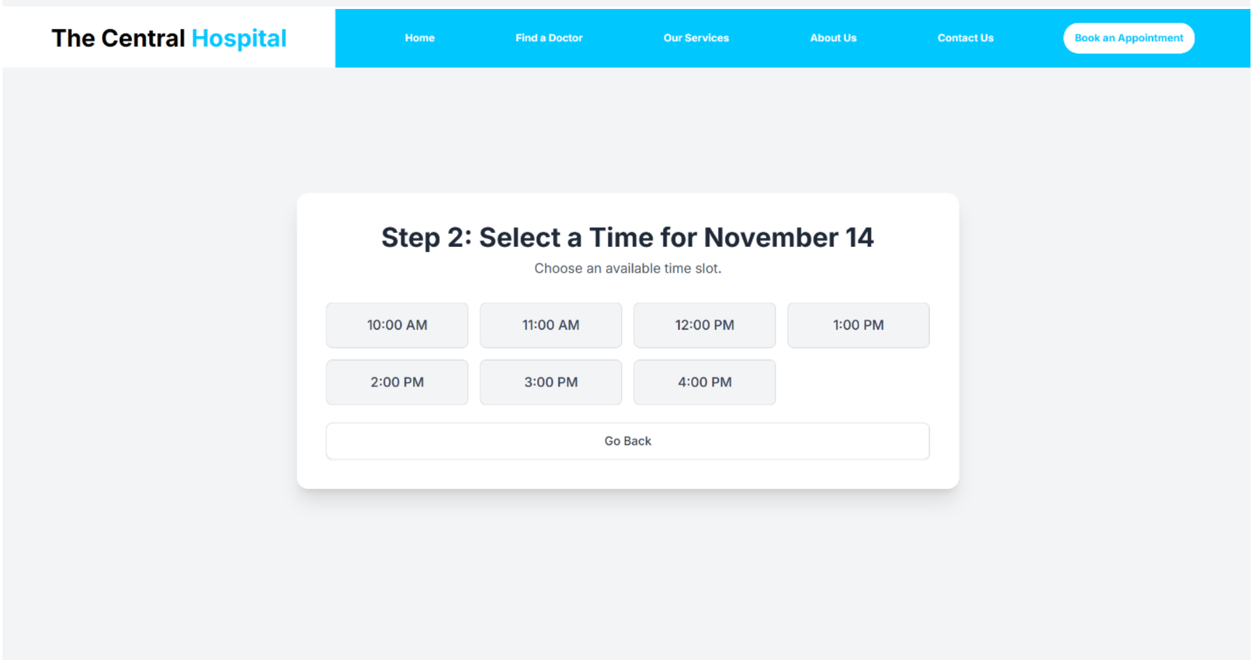

The Central Hospital
Home
Find a Doctor
Our Services
About Us
Contact Us
Book an Appointment
Send Us a Message
First name:
Last name:
Enter your email:
Enter your phone number:
What is the reason for your message?
General Inquiry
Customer Support & Help Request
Sales Inquiry
Feedback & Suggestions
Your Message:
Send

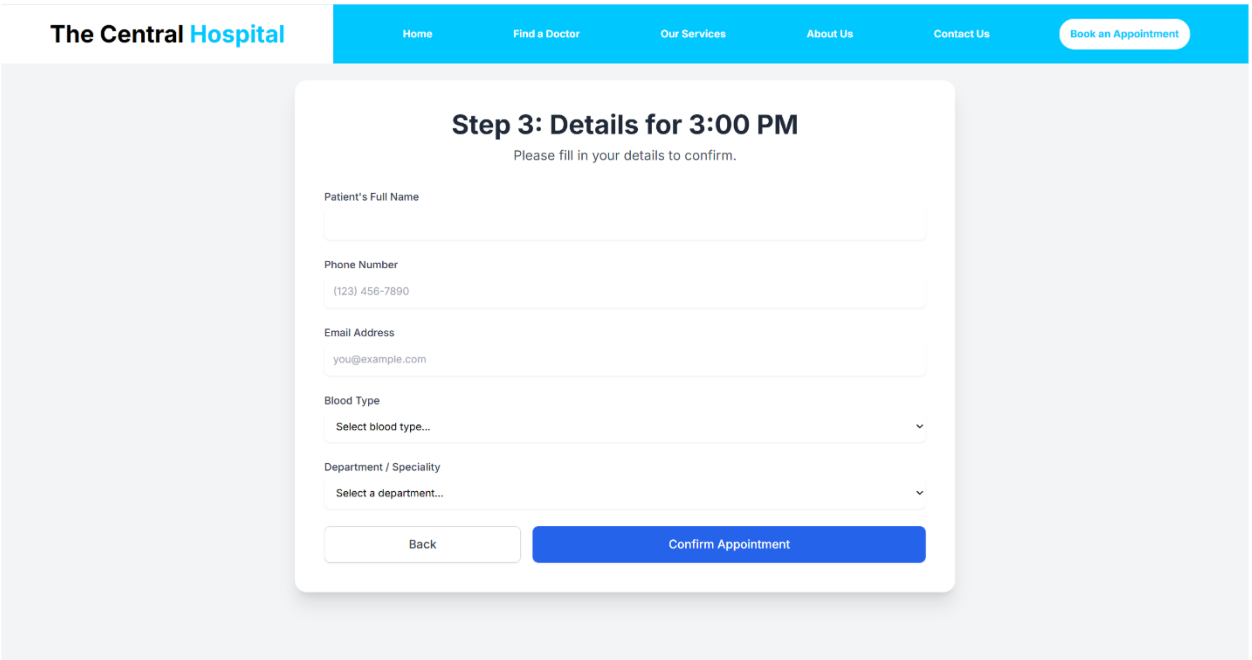
3.7 Appointment Booking System (BookAnAppointment.html)

This is the most technically complex page in the project.

- **Technology Stack:** This page uniquely uses **Tailwind CSS** for UI components, which is imported via a CDN.
- **Multi-Step UI:** The JavaScript logic manages a 3-step process by hiding and showing different div elements (booking-calendar-step, booking-time-step, patient-form-step).
- **Step 1: Select Date:** A calendar grid is dynamically generated by the buildCalendarGrid() function.
- **Step 2: Select Time:** Based on the selected date, the buildTimeSlotsForDate() function generates available time slots. It checks localStorage (simulating a database) to see if a slot is "reserved" and disables it if necessary.
- **Step 3: Patient Details:** A standard HTML form for collecting patient information.
- **Data Submission:** On form submission, an event listener prevents the default action. It gathers all data (selected date, time, and form details) and uses the **fetch() API** to send it to book_appointment.php.
- **Confirmation:** The script then receives a plain text response from the PHP script. If the response is "Appointment booked successfully!", it displays a confirmation message to the user.


The Central Hospital
Home
Find a Doctor
Our Services
About Us
Contact Us
Book an Appointment
Book an Appointment
Step 1: Select a Date
~~November 10~~
November 11
November 12
November 13
November 14
November 15
November 16
November 17
November 18
November 19
November 20
November 21
November 22
November 23
November 24
~~November 25~~
~~November 26~~
~~November 27~~
~~November 28~~
~~November 29~~


The Central Hospital
Home
Find a Doctor
Our Services
About Us
Contact Us
Book an Appointment
Step 2: Select a Time for November 14
Choose an available time slot.
10:00 AM
11:00 AM
12:00 PM
1:00 PM
2:00 PM
3:00 PM
4:00 PM
Go Back


The Central Hospital
Home
Find a Doctor
Our Services
About Us
Contact Us
Book an Appointment
Step 3: Details for 3:00 PM
Please fill in your details to confirm.
Patient's Full Name
Phone Number
(123) 456-7890
Email Address
you@example.com
Blood Type
Select blood type...
Department / Speciality
Select a department...
Back
Confirm Appointment

4. Backend Implementation: PHP & MySQL

4.1 Database Connection

Both PHP scripts use standard mysqli to connect to a local MySQL database with the credentials:

- **Server:** localhost
- **Username:** root
- **Password:** "" (empty)
- **Database:** central_hospital

The scripts also include CREATE DATABASE IF NOT EXISTS and CREATE TABLE IF NOT EXISTS queries, making them robust and capable of self-initializing the required database structure on the first run.

4.2 Contact Form Handler (contact.php)

This script handles submissions from SendUsAMessage.html.

1. **Receives Data:** It accesses the $_POST superglobal to get the form data (fname, lname, email, etc.).
2. **Validation:** It performs a simple empty() check on required fields.
3. **Database Interaction:** It uses a **prepared statement** ($stmt = $conn->prepare(...)) to insert the data. This is a critical security measure to prevent **SQL injection**.
4. **Feedback:** After execution, it sets a $userMessage variable.
5. **Redirect:** The script (which is a mix of PHP and HTML) then embeds this $userMessage into a JavaScript alert() and uses window.location.href to redirect the user back to the contact form page.

4.3 Appointment Booking Handler (book_appointment.php)

This script is designed to work as an API endpoint for the fetch() request from BookAnAppointment.html.

1. **Receives Data:** It's more complex than the contact script. It checks $_POST first, but also attempts to read php://input to handle different content types (like application/x-www-form-urlencoded from the fetch request).
2. **Database Interaction:** It also uses prepared statements (bind_param) to securely insert the appointment details into the appointments table.
3. **Response:** Instead of redirecting, it simply uses echo to send a plain text string back to the client.
    - On success: echo "Appointment booked successfully!";
    - On failure: echo "Error: ..."; This string is what the client-side JavaScript reads in its .then(data => ...) block to confirm success or failure to the user.

5. Project Challenges

- **CSS Consistency:** The project started with internal CSS and later introduced Tailwind CSS for one page. This led to an inconsistent design language and difficulties in maintenance. A global, external stylesheet would have been a better approach.
- **Client-Side State:** Managing the state of the "Find a Doctor" search and the "Booking" wizard purely with JavaScript was complex.
- **Booking Logic:** The localStorage solution for tracking booked slots is a simulation. In a real-world application, this would fail as it's not shared between users. The backend would need to be queried in real-time to check for slot availability.
- **PHP Data Handling:** Ensuring the book_appointment.php script could correctly parse the data sent from the fetch API (which was URL-encoded) required extra debugging and data-handling logic.

6. Conclusion and Future Work

This project successfully demonstrates the creation of a full-stack web application. It combines static information delivery with dynamic, interactive features like search and booking, and securely persists user data in a MySQL database.

The final product is a functional prototype that meets all the core objectives. The separation of frontend logic (JS) and backend processing (PHP) represents a clear understanding of modern web development principles.

Future Work:

- **Admin Panel:** Create a secure, login-protected admin dashboard where hospital staff can view and manage all messages and appointments stored in the database.
- **Real-Time Booking:** Replace the localStorage logic with a real-time system. Before showing time slots, the JavaScript should fetch from a PHP script to get *only* the truly available slots from the database.
- **User Authentication:** Implement a patient login system where users can create accounts, view their past and upcoming appointments, and manage their profiles.
- **Refactor CSS:** Migrate all internal styles to one or two external .css files to ensure consistency and improve performance.